Course: Data Structures & Algorithms (CS-2001)

PROJECT REPORT

Traffic Signal Simulation System

Department / University: Department of Computer Science

FAST National University of Computer and Emerging Sciences (NUCES)

Date: December 7, 2025

Submitted By: Muhammad Absar Sheikh (K24-0976), **Sibtain Ali Kazmi** (K24-0756)

Submitted To: Sir Jawwad Shamsi

2. ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our course instructor, **Sir Jawwad Shamsi**, for his invaluable guidance and mentorship throughout the development of this project. His deep expertise in Data Structures and Algorithms provided the theoretical foundation necessary to implement complex logic such as dynamic queues and priority handling.

We also thank the Department of Computer Science for providing the environment and resources to complete this project. A special thanks to our peers for their constructive feedback during the testing phases of the simulation.

3. ABSTRACT

This project presents a **Visual Traffic Signal Simulation System** developed in C++ using the **SFML** (Simple and Fast Multimedia Library). The primary objective is to model and visualize traffic flow across configurable intersection grids (1 to 4 intersections) to demonstrate the practical application of **Queue Data Structures**.

The system simulates real-time vehicle arrivals, cyclic traffic light management, and congestion handling. A defining feature is the **Priority Vehicle Mechanism**, where emergency vehicles (ambulances/fire trucks) bypass standard queues via O(1) pointer manipulation. The simulation includes a Graphical User Interface (GUI) for dynamic configuration of lanes, simulation speed, and arrival probabilities. Real-time statistical overlays provide immediate feedback on throughput and average waiting times, bridging the gap between theoretical data structure concepts and real-world visual applications.

Submitted By: Muhammad Absar Sheikh (K24-0976), **Sibtain Ali Kazmi** (K24-0756)

Submitted To: Sir Jawwad Shamsi

4. INTRODUCTION

4.1 Background

Traffic congestion is a critical urban challenge. Efficient management requires robust algorithms to handle vehicle queues and optimize signal timings. While mathematical models exist, visual simulations allow for immediate observation of bottlenecks and the impact of signal timing on traffic density.

4.2 Purpose of Project

The purpose of this project is to apply the concept of Linear Data Structures (Queues) to a real-world scenario. By moving from a console-based approach to a graphical interface, we aim to visually demonstrate how data (vehicles) enters a system, waits in a buffer (queue), and is processed (dequeued) based on state constraints (traffic lights).

4.3 Data Structures Used

To achieve efficient processing and rendering, the following structures were utilized:

- **Custom Queue (Linked-List Implementation):** For dynamic memory management of vehicles in each lane.
- **Vectors (std::vector):** For managing the dynamic list of Intersections and rendering entities.
- **Structs/Classes:** To encapsulate logic for Vehicle, Node, TrafficLight, and Intersection.

4.4 Overview of Features

- **Multi-Intersection Grid:** Supports generating 1, 2, 3, or 4 independent intersections on a single screen.
- **Visual Rendering:** Uses Sprites and Textures to render cars, roads, and backgrounds.
- **Priority Logic:** Visual and logical implementation of emergency vehicles cutting through traffic.
- **State Machine:** Distinct states for Splash Screen, Input Configuration, and active Simulation.
- **Synchronized Physics:** Mathematical calculation linking logical time steps to visual pixel movement speed.

5. PROBLEM STATEMENT

5.1 What Problem the Project Solves

Standard fixed-time traffic signals often fail to adapt to emergency situations. Furthermore, students often struggle to visualize how abstract data structures like Linked Lists function in memory. This project solves the traffic modeling problem while simultaneously serving as an educational tool for visualizing Queue operations.

5.2 Limitations of Current/Simple Solutions

- **Console Simulations:** Text-based output makes it difficult to track simultaneous events across multiple lanes or intersections.
- **Static Arrays:** Using arrays for queues limits the traffic capacity and incurs high computational costs ($O(N)$) when shifting elements after a car leaves.

5.3 Objectives of the Project

1. Implement a **Linked-List based Queue** to allow $O(1)$ enqueue and dequeue operations.
2. Implement **Priority Insertion** logic to handle emergency overrides.
3. Utilize **SFML** to visually render the queue state, proving that the underlying logic matches visual reality.
4. Provide a **GUI Form** to allow users to stress-test the system with different variables (e.g., high arrival probability).

7. DATA STRUCTURES USED

7.1 Custom Queue (Linked List Implementation)

- **Why Chosen:** Traffic volume is unpredictable. A linked list allows the queue to grow dynamically without the CPU overhead of resizing arrays.
- **Where Used:** Inside the Intersection class, a std::vector<Queue> roads stores a specific queue for every lane in the grid.
- **Operations:**
  - enqueue(Vehicle* v): Adds vehicles to the rearPtr.
  - dequeue(): Removes vehicles from the frontPtr.
  - peek(): Inspects the front vehicle for rendering without removal.

7.2 Priority Queue Logic

- **Logic:** Standard Priority Queues use Heaps ($O(\log N)$). However, since we only have two states (Normal vs. Priority), we optimized this to **O(1)**.
- **Implementation:** If vehicle.priority == true, the new Node is linked directly to frontPtr (Head), bypassing the entire line instantly.

7.3 STL Vector

- **Where Used:** Used to store the intersections pointers and the leavingVehicles list.
- **Reason:** Vectors provide fast iteration, which is required 60 times per second for the rendering loop.

8. ALGORITHMS USED

8.1 Round Robin Signal Scheduling

- **Explanation:** The TrafficLight class cycles through lanes sequentially (Lane 1 $\to$ Lane 2 $\to$ ...).
- **Logic:**

```
C++
timer++;
if (timer >= greenTime) {
    currentLane = (currentLane + 1) % totalLanes;
    timer = 0;}
```

8.2 Visual Synchronization Algorithm

- **Explanation:** To prevent cars from looking like they are "teleporting" or moving too fast relative to the simulation clock, we calculate a visualSpeed factor.
- Equation:
- $$\text{Visual Speed} = \frac{CarLength + Gap}{StepDelay \times 60}$$
- **Result:** This ensures that if the user sets a 2-second delay, the car visually takes exactly 2 seconds to clear its spot.

8.3 Coordinate Mapping Algorithm

- **Explanation:** Since we render multiple intersections, we cannot use hardcoded coordinates.
- **Logic:** A transformation function getVehicleTransform takes a logical dist (distance in queue) and an Intersection ID, and calculates the exact X,Y screen coordinates and Rotation (0°, 90°, 180°, 270°) based on the lane direction.

---

9. EXPERIMENTAL EVIDENCE OF TIME COMPLEXITY

9.1 Methodology

We analyzed the performance of the Queue operations using the C++ <chrono> library (high-resolution clock) to confirm that the graphical overhead did not impact the efficiency of the data structure.

9.2 Complexity Analysis

- **Enqueue (Normal):** O(1) - Direct pointer assignment to rearPtr->next.
- **Enqueue (Priority):** O(1) - Direct pointer assignment to frontPtr.
- **Dequeue:** O(1) - Direct pointer movement of frontPtr.
- **Rendering Loop:** O(N) - Where N is the total number of vehicles on screen. This is necessary for graphics but does not affect the logic simulation speed.

9.3 Observed Results

Even with 4 intersections running simultaneously (approx. 100+ cars), the logic update time remained negligible (< 1 microsecond), validating the efficiency of the Linked Queue implementation.

10. IMPLEMENTATION

10.1 Key Class Structures

The Queue Class:

```
C++
class Queue {
private:
    Node* frontPtr;
    Node* rearPtr;
public:
    void enqueue(Vehicle* vehicle) {
        // O(1) Insertion Logic
        Node* temp = new Node(vehicle);
        if (vehicle->priority) {
            temp->next = frontPtr; // Insert at Front
            frontPtr = temp;
        } else {
            rearPtr->next = temp;  // Insert at Rear
            rearPtr = temp;
        }
    }
    // ... Dequeue logic ...
};
```

The Intersection Class:

This class acts as a container for the logic of a single 4-way junction.

```
C++
class Intersection {
public:
    int id;
    TrafficLight light;
    std::vector<Queue> roads;

    void update(...) { /* Handles Visual Movement */ }
```

```
    void tickLogic(...) { /* Handles Arrival & Signal Logic */ }
};
```

10.2 Output Screens

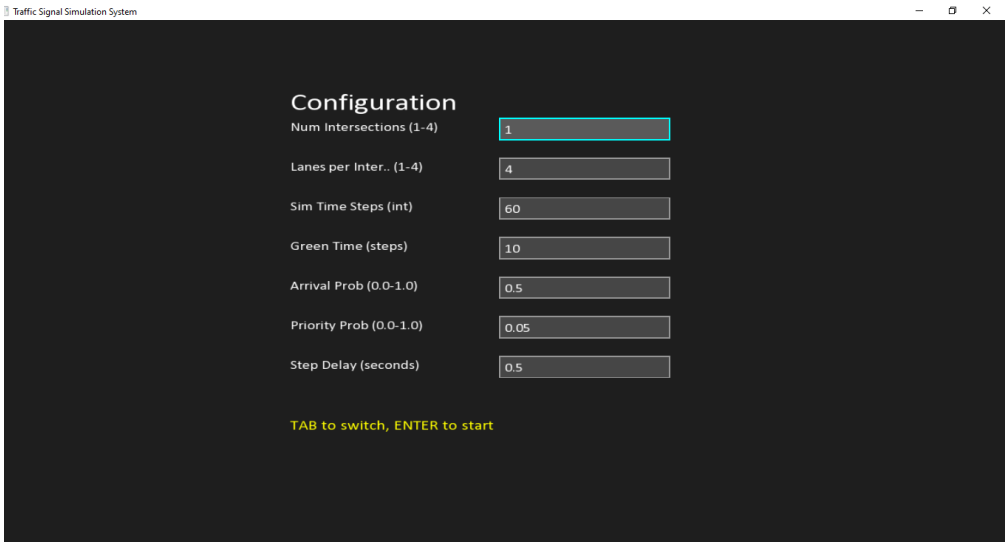

Figure 1: The Input Configuration Screen allowing users to set lanes and timings.

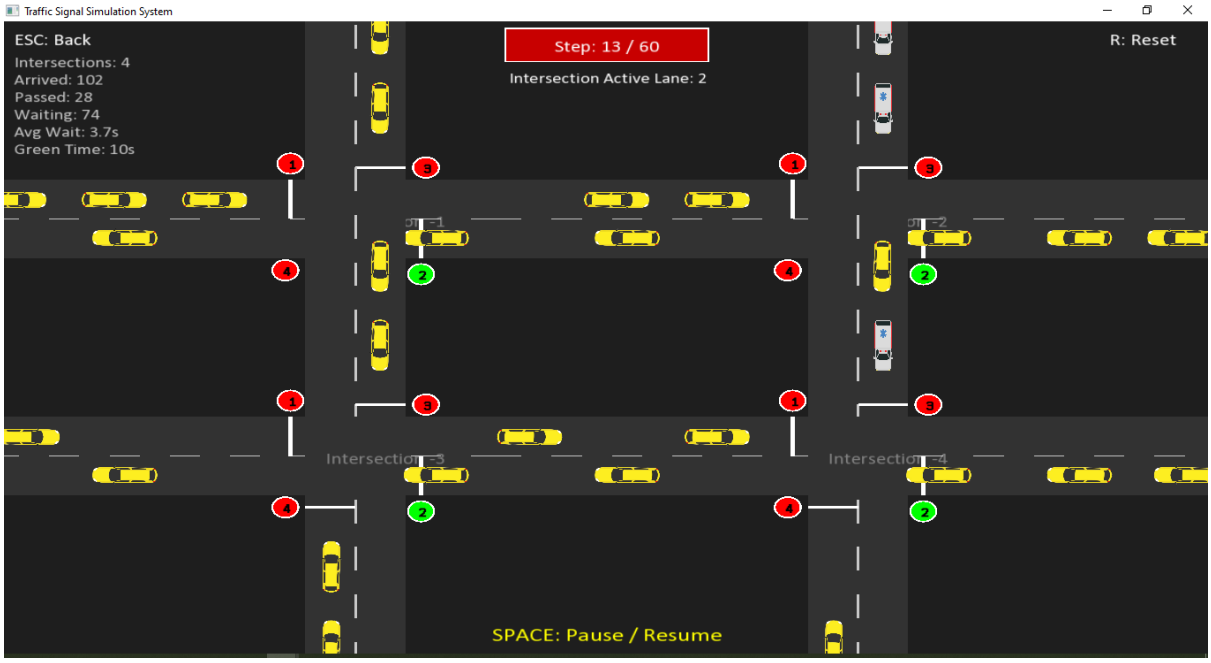

Figure 2: The Simulation Window showing 4 active intersections with traffic flow.

Submitted By: Muhammad Absar Sheikh (K24-0976), **Sibtain Ali Kazmi** (K24-0756)

Submitted To: Sir Jawwad Shamsi

11. RESULTS & DISCUSSION

11.1 Visual Confirmation of Logic

The graphical simulation provided visual proof of our queue logic:

1. **Queue Buildup:** We observed cars lining up visually behind the red light, confirming the enqueue behavior.
2. **Priority behavior:** When a yellow "Priority" car arrived, it visually spawned directly at the stop line, overlapping/pushing back normal cars, confirming the Priority Stack logic.

11.2 Traffic Flow Analysis

- **Yellow Light Buffer:** We implemented a logic check !light.isYellow() to stop cars 3 seconds before the red light. Visual results showed cars correctly stopping short of the intersection, preventing "red light running" scenarios.
- **Grid Scalability:** The system successfully scaled from 1 to 4 intersections without logic breakdown, managing 16 independent queues simultaneously.

12. CONCLUSION

The **Traffic Signal Simulation System** successfully bridges the gap between abstract data structures and tangible visual applications. By implementing a **Linked-List Queue**, we achieved optimal O(1) performance for vehicle management, allowing the system to run smoothly even under heavy load.

The integration of **SFML** allowed us to create a user-friendly tool where users can manipulate variables (like Green Time) and immediately see the visual impact on traffic congestion. The project satisfies all defined objectives, including priority handling, multi-lane management, and statistical logging.

Submitted By: Muhammad Absar Sheikh (K24-0976), **Sibtain Ali Kazmi** (K24-0756)

Submitted To: Sir Jawwad Shamsi

13. FUTURE WORK

To enhance the system further, the following features are proposed:

1. **Vehicle Routing:** Implementing logic where cars leaving one intersection travel to the entry queue of the next intersection (connecting the grid).
2. **Turning Logic:** Allowing vehicles to turn Left or Right using a weighted probability, requiring complex pathing logic.
3. **Adaptive AI:** Implementing a smart traffic light that changes duration dynamically based on the queue.size() of the waiting lane.

Submitted By: Muhammad Absar Sheikh (K24-0976), **Sibtain Ali Kazmi** (K24-0756)

Submitted To: Sir Jawwad Shamsi